Gemini Voice
Master Resource Guide

Building Real-Time Voice Agents with Google Gemini 3.1 Flash Live
Two Complete Demos: Aria Executive Assistant + E-Commerce Voice Widget

Built with Claude Code • Gemini 3.1 Flash Live • FastAPI • Web Audio API
google-genai SDK • ClickUp API • Google Calendar API

Table of Contents

1. Project Overview

2. Gemini 3.1 Flash Live — The Model

3. Platform Comparison — Gemini vs OpenAI vs ElevenLabs

4. Project Structure — Full File Map

5. Demo 1: Aria — Executive Assistant (aria-assistant/)

6. Demo 2: Apex Keyboards — Sales Widget (keyboard-demo/)

7. Architecture Deep Dive — How It All Works

8. Setup Guide — From Zero to Running

9. How to Add Your Own Tools

10. What to Tell Claude Code

11. Deployment to Production

12. Known Issues & Tips

1. Project Overview

This project contains two complete demos showing how to build real-time voice agents with Google's Gemini 3.1 Flash Live API. Both use the same core architecture: a Python FastAPI server acts as a WebSocket relay between a browser and Gemini's Live API, streaming audio bidirectionally in real-time.

The Two Demos

Demo 1: Aria — Executive Assistant

A voice-powered receptionist that manages your Google Calendar and ClickUp tasks through natural conversation. Say things like "What's on my calendar today?" or "Create a task called Review Q2 Report." Aria uses server-side tool calling — Gemini decides when to call a function, your server executes it, and sends the result back. This is the more advanced demo.

Demo 2: Apex Keyboards — Sales Widget

A dark-themed e-commerce landing page for a fictional keyboard brand with a floating mic button. Visitors can ask about products, shipping, and return policies. The agent's knowledge comes from its system prompt — no tool calling needed. This is the simpler demo, focused on showing how to embed a voice widget on any website.

Why Native Speech-to-Speech Matters

Most voice assistants use a 3-step pipeline: Speech-to-Text → LLM → Text-to-Speech. Each step adds latency and loses information (tone, emotion, hesitation). Gemini 3.1 Flash Live is natively multimodal — it processes audio directly and generates audio directly.

- Sub-second response times (no pipeline overhead)
- The model hears tone, emphasis, and emotion — not just words
- Natural interruption support (barge-in) — talk over it and it stops immediately
- No separate STT or TTS service — just one API

Architecture (Both Demos)

```
Browser (mic + speaker)
    |  WebSocket (PCM audio as base64 JSON)
    v
FastAPI server (Python)
    |  google-genai SDK (WebSocket)
    v
Gemini 3.1 Flash Live API
    |  tool calls (Aria only)
    v
Python tool functions (ClickUp, Calendar)
```

2. Gemini 3.1 Flash Live — The Model

What's New in 3.1 Flash Live

- Google's highest-quality audio model for real-time dialogue
- Better tonal understanding — more natural, expressive responses
- Can follow conversation threads for 2x longer than previous models
- Better at filtering background noise (traffic, TV, etc.)
- Faster response times than 2.5 Flash
- 90+ languages supported with real-time language switching
- Full-duplex conversation — users can interrupt mid-sentence (barge-in)
- All audio output is watermarked to prevent misuse
- Model ID: gemini-3.1-flash-live-preview

Session Limits

Dimension	Limit
Context window	128k tokens
Audio-only session	15 minutes max
Audio + video session	2 minutes max
Concurrent sessions (pay-as-you-go)	Up to 1,000 per project

Pricing

Audio is converted at 25 tokens per second for both input and output.

Model	Input (per 1M tokens)	Output (per 1M tokens)
Gemini 3.1 Flash	$0.50	$3.00
Gemini 3.1 Flash-Lite	$0.25	$1.50
Gemini 3.1 Pro	$2.00	$12.00

Rough cost: A 5-minute voice conversation ≈ 7,500 input tokens + 7,500 output tokens. At Flash pricing: ~$0.03 per conversation. A free tier is available through Google AI Studio with rate limits.

How to Get an API Key

1. Go to https://aistudio.google.com/apikey
2. Sign in with your Google account
3. Click 'Create API Key' and select or create a Google Cloud project
4. Copy the key — you'll put this in your .env file as GOOGLE_API_KEY

3. Platform Comparison

	Gemini 3.1 Flash Live	OpenAI Realtime API	ElevenLabs Conv. AI
Architecture	Native speech-to-speech	Native speech-to-speech	STT → LLM → TTS pipeline
Deploy complexity	Need a WebSocket server ($5-7/mo)	Need a WebSocket server	One <script> tag (hosted)
Audio input cost	~$0.50/1M tokens (25 tok/sec)	$0.06/min + $10/1M tokens	$0.10/min (agent time)
Audio output cost	~$3.00/1M tokens (25 tok/sec)	$0.24/min + $20/1M tokens	Included
5-min conversation cost	~$0.03	~$1.50+	~$0.50
Voice quality	Good (native)	Good (native)	Excellent (best TTS)
Latency	Very low	Very low	Low (~200ms pipeline)
Interruption	Yes (native)	Yes (native)	Yes
Tool calling	Yes (synchronous)	Yes (deterministic JSON)	Limited
Languages	90+	~30	30+
Voice cloning	No	No	Yes
Free tier	Yes (rate limited)	No	No (trial only)

When to Use Each

- Gemini 3.1 Flash Live — Best value. Cheapest per conversation. Full tool calling. Best choice if you want control and don't mind a small server.
- OpenAI Realtime API — Best structured tool calling. Higher cost but great DX. Use if you're in the OpenAI ecosystem.
- ElevenLabs — Easiest to deploy (one script tag). Best voice quality and voice cloning. Use if you want zero backend work.

4. Project Structure — Full File Map

```
Gemini Voice/
├── .env                        # API keys (GOOGLE_API_KEY, CLICKUP_API_KEY)
├── CLAUDE.md                   # Instructions for Claude Code
├── README.md                   # Project overview & setup
│
├── aria-assistant/             # Demo 1: Executive Assistant
│   ├── server.py               # FastAPI server + Gemini relay + tool execution
│   ├── index.html              # Single-file browser UI (mic, transcripts, tool
indicator)
│   ├── requirements.txt        # Python dependencies
│   ├── client_secret_*.json    # Google OAuth credentials (not committed)
│   ├── token.json              # OAuth token (auto-generated on first run)
│   └── tools/
│       ├── __init__.py
│       ├── clickup.py          # ClickUp API: workspaces, spaces, lists, tasks
│       └── calendar.py         # Google Calendar: events, create, details
│
└── keyboard-demo/              # Demo 2: E-Commerce Voice Widget
    ├── server.py               # FastAPI server + Gemini relay (no tools)
    ├── requirements.txt        # Python dependencies
    └── public/
        ├── index.html          # Landing page HTML
        ├── css/styles.css      # Dark theme, animations, voice widget styles
        └── js/
            ├── products.js     # Product catalog data + card renderer
            ├── audio-recorder.js # Mic capture → PCM16 base64
            ├── audio-streamer.js # PCM16 base64 → speaker playback
            ├── gemini-live-client.js # WebSocket client to local server
            └── voice-widget.js # Mic button UI + state machine
```

Shared: .env File

Both demos read from the same .env file in the Gemini Voice/ root. Only GOOGLE_API_KEY is required for the keyboard demo. Aria also needs CLICKUP_API_KEY.

```
GOOGLE_API_KEY=your-gemini-api-key-here
CLICKUP_API_KEY=your-clickup-api-key    # only for Aria
```

Shared: CLAUDE.md

This file gives Claude Code context about the project when working on it. It describes the architecture, how to run each demo, the audio pipeline, and the tool execution pattern. If you open this project in Claude Code, it reads CLAUDE.md automatically.

5. Demo 1: Aria — Executive Assistant

Aria is a voice-powered receptionist that manages your schedule and tasks. It demonstrates the full power of Gemini's Live API including real-time tool calling.

What Aria Can Do

- "What's on my calendar today?" — reads your Google Calendar
- "Schedule a meeting for Friday at 2pm" — creates calendar events
- "What workspaces do I have in ClickUp?" — browses your ClickUp
- "Create a task called Review Q2 Report" — creates ClickUp tasks
- "Update that task to high priority" — modifies existing tasks
- "Close the task" — marks tasks as complete

aria-assistant/server.py

The core of Aria. A FastAPI app with a /ws WebSocket endpoint. On each connection it:

5. Creates a Gemini Live session with LiveConnectConfig (model, system prompt, tools, transcription)
6. Runs two concurrent async tasks: browser_to_gemini() and gemini_to_browser()
7. browser_to_gemini(): receives base64 audio from the browser, decodes it, calls session.send_realtime_input(audio=Blob(...))
8. gemini_to_browser(): receives Gemini responses and forwards audio, transcripts, turn_complete, and interrupted events
9. When Gemini sends a tool_call: executes the Python function from the TOOLS dict, sends the result back via session.send_tool_response()

Key configuration in server.py:

Setting	Value
Model	gemini-3.1-flash-live-preview
Port	8000
Response modalities	["AUDIO"]
System prompt	~60 lines defining Aria's personality, capabilities, and guidelines
Tools	13 function declarations (9 ClickUp + 4 Calendar)
Transcription	Enabled for both input and output

aria-assistant/index.html

A single-file browser client (no build step). Contains all HTML, CSS, and JavaScript. Key features:

- Mic capture via AudioWorklet (inline data: URL — no separate worklet file)
- PCM16 encoding at 16kHz, sent as base64 JSON over WebSocket
- Audio playback queue with sequential scheduling at 24kHz
- clearPlayback() for instant barge-in (empties queue, stops playback)
- Real-time transcript display (user messages right-aligned, agent left-aligned)

- Tool call indicator with spinner (shows “Checking today’s calendar...” etc.)

aria-assistant/tools/clickup.py

ClickUp API v2 wrapper. 9 functions that all return {status: success/error, ...} dicts. Uses the CLICKUP_API_KEY from .env. Functions:

Function	What It Does
get_workspaces()	Lists all workspaces (teams)
get_spaces(team_id)	Lists spaces in a workspace
get_lists(space_id)	Lists all task lists (including inside folders)
get_tasks(list_id, status?)	Gets tasks from a list, optional status filter
get_task_details(task_id)	Full details of one task
create_task(list_id, name, ...)	Creates a new task
update_task(task_id, status?, priority?)	Updates status or priority
add_comment(task_id, comment_text)	Adds a comment to a task
close_task(task_id)	Closes/completes a task

aria-assistant/tools/calendar.py

Google Calendar API wrapper using OAuth 2.0. Requires a client_secret JSON file from Google Cloud Console. On first run, opens a browser for Google login and saves token.json. After that, tokens auto-refresh. 4 functions:

Function	What It Does
get_todays_events()	All events for today
get_upcoming_events(days?)	Events for the next N days (default 7)
get_event_details(event_id)	Full details of one event
create_event(summary, start, end, ...)	Creates a new calendar event

How Tool Calling Works

This is the key pattern. Tools are registered in two parallel structures in server.py:

10. TOOLS dict — maps function name strings to Python callables: {'get_workspaces': clickup.get_workspaces, ...}
11. TOOL_DECLARATIONS list — JSON schema descriptions sent to Gemini so it knows what tools exist and what parameters they take
12. When you speak, Gemini decides if a tool is needed based on your request
13. Gemini pauses speaking and sends a tool_call message with function name + args
14. Server executes the function via execute_tool(name, args)
15. Server sends the result back via session.send_tool_response(function_responses=[...])
16. Gemini resumes speaking, incorporating the result into its response

Tool calls are synchronous — the model pauses while your function runs. The functions themselves use the requests library (synchronous HTTP), called from async context.

6. Demo 2: Apex Keyboards — Sales Widget

A simpler demo focused on embedding a voice widget on a website. No tool calling — the agent's knowledge comes entirely from its system prompt.

What It Does

- Answers questions about 4 mechanical keyboards ($109–$199)
- Recommends products based on use case (gaming, typing, portability)
- Explains shipping options, return policy, and warranty
- Conversational only — users click "Add to Cart" on the page themselves

keyboard-demo/server.py

Same FastAPI relay pattern as Aria, but simpler — no tools. The system prompt contains the full product catalog (4 keyboards with specs, prices, and recommendations), shipping tiers, and conversation guidelines. Port 3001.

keyboard-demo/public/ — The Frontend

Unlike Aria's single-file approach, this demo splits the frontend into separate files:

File	What It Does
index.html	Page structure: nav, hero section, product grid, shipping banner, footer, voice widget mount point
css/styles.css	763 lines. Dark theme (#0a0a0f), purple accent (#6c5ce7), product cards with hover effects, mic button with pulse/spinner animations, responsive at 768px and 480px
js/products.js	Product data (window.PRODUCTS array, window.SHIPPING_INFO object) + DOM renderer that builds product cards on page load
js/audio-recorder.js	AudioRecorder class: getUserMedia at 16kHz, ScriptProcessorNode, Float32→Int16→base64. Echo cancellation + noise suppression enabled.
js/audio-streamer.js	AudioStreamer class: decodes base64→Int16→Float32, schedules playback at 24kHz with look-ahead. clearQueue() for instant barge-in.
js/gemini-live-client.js	WebSocket client connecting to ws://localhost:3001/ws. Sends {type: 'audio', data} messages. Exposes callbacks: onready, onaudio, onturncomplete, oninterrupted, etc.
js/voice-widget.js	UI controller. Injects mic button into #voice-widget-container. State machine: idle → connecting → listening → agent_speaking. Wires all other JS classes together.

7. Architecture Deep Dive

Why a Server Relay (Not Direct Browser → Gemini)?

- The google-genai SDK is Python-only. Raw WebSocket from the browser has protocol quirks that cause silent failures and hangs.
- No browser-safe auth — you'd expose your API key in page source.
- The SDK handles connection management and the Gemini protocol correctly.
- Server-side tool execution is more secure (API keys for ClickUp/Calendar stay on the server).

Audio Pipeline (Step by Step)

```
 1. User speaks into microphone
 2. Browser captures via getUserMedia() at 16kHz mono
 3. Audio processor converts Float32 → Int16 PCM → base64
 4. Browser sends {type: 'audio', data: '<base64>'} via WebSocket
 5. Server decodes base64 → raw bytes
 6. Server calls session.send_realtime_input(
        audio=types.Blob(data=bytes, mime_type='audio/pcm;rate=16000')
    )
 7. Gemini processes audio natively (no STT step)
 8. Gemini generates audio response natively (no TTS step)
 9. Server receives audio from session.receive()
10. Server base64-encodes → sends {type: 'audio', data: '<base64>'}
11. Browser decodes base64 → Int16 → Float32, schedules playback at 24kHz
12. User hears the response
```

Barge-In (Interruption) Flow

17. User starts speaking while agent audio is playing
18. Gemini's built-in Voice Activity Detection (VAD) detects the user's voice
19. Gemini stops generating and sends an 'interrupted' message
20. Server relays {type: 'interrupted'} to browser
21. Browser clears the audio playback queue (instant silence)
22. UI state returns to 'listening' — ready for the user's new input

WebSocket Protocol (Browser ↔ Server)

Direction	Message	Purpose
Browser → Server	{"type": "audio", "data": "<base64>"}	Mic audio chunk
Server → Browser	{"type": "audio", "data": "<base64>"}	Agent audio response
Server → Browser	{"type": "status", "message": "Connected to Gemini"}	Session ready
Server → Browser	{"type": "input_transcript", "text": "..."}	What the user said
Server → Browser	{"type": "output_transcript", "text": "..."}	What the agent said
Server → Browser	{"type": "turn_complete"}	Agent finished speaking
Server → Browser	{"type": "interrupted"}	User barged in

Server → Browser	{"type": "tool_call", "name": "..."}	Tool being executed (Aria)
Server → Browser	{"type": "tool_result", "name": "...", "status": "..."}	Tool finished (Aria)
Server → Browser	{"type": "error", "message": "..."}	Error occurred

Critical Implementation Detail: send_realtime_input

When sending audio to Gemini, you MUST use the audio= parameter:

```
# CORRECT — works with Gemini 3.1 Flash Live
await session.send_realtime_input(
    audio=types.Blob(data=audio_bytes, mime_type='audio/pcm;rate=16000')
)

# WRONG — deprecated, Gemini 3.1 rejects it
await session.send_realtime_input(
    media=types.Blob(data=audio_bytes, mime_type='audio/pcm;rate=16000')
)
```

8. Setup Guide — From Zero to Running

Prerequisites

- Python 3.10+
- A Google AI API key (free tier works) — https://aistudio.google.com/apikey
- A microphone
- Chrome or Edge browser
- For Aria: A ClickUp account + API key (https://app.clickup.com/settings/apps)
- For Aria's Calendar: Google Cloud OAuth credentials (Desktop app type)

Step 1: Create .env

In the Gemini Voice/ root directory:

```
GOOGLE_API_KEY=your-gemini-api-key
CLICKUP_API_KEY=your-clickup-api-key    # only for Aria
```

Step 2: Install Dependencies

```
# For the keyboard demo:
cd keyboard-demo
pip install -r requirements.txt

# For Aria (includes Google Calendar OAuth libs):
cd aria-assistant
pip install -r requirements.txt
```

Step 3: Google Calendar Setup (Aria Only)

23. Go to Google Cloud Console → APIs & Services → Credentials
24. Click + CREATE CREDENTIALS → OAuth Client ID → Desktop app
25. Download the JSON file into the aria-assistant/ folder
26. On first run, a browser window opens for Google login
27. After login, token.json is saved automatically — no login needed again

Step 4: Run a Demo

```
# Aria (port 8000):
cd aria-assistant
python server.py
# Open http://localhost:8000

# Keyboard demo (port 3001):
cd keyboard-demo
python server.py
# Open http://localhost:3001
```

Step 5: Talk to It

28. Click the mic button
29. Grant microphone permission when prompted

30. Wait for ‘Connected’ / ‘Listening...’ status
31. Start talking naturally
32. Click the mic button again to stop

Troubleshooting

Problem	Cause	Fix
‘Connecting’ spins forever	Wrong model or API key	Check server console. Verify GOOGLE_API_KEY.
‘Session ended’ immediately	Model doesn’t support bidiGenerateContent	Use gemini-3.1-flash-live-preview
No audio playback	Browser autoplay policy	Must click mic button first (user gesture)
Mic denied	Browser blocked access	Check browser settings, use localhost or HTTPS
Echo / feedback	Speakers picked up by mic	Use headphones
Port in use	Another service running	Change port in server.py or kill other process
Calendar 403 error	OAuth not set up	Follow Step 3 above for Google Calendar setup

9. How to Add Your Own Tools

The Aria demo shows the pattern for adding tools. To add a new tool (e.g., a weather API):

Step 1: Create the Python function

```
# tools/weather.py
import requests

def get_weather(city: str) -> dict:
    try:
        resp = requests.get(f"https://api.weather.com/v1/{city}")
        return {"status": "success", "temperature": resp.json()["temp"]}
    except Exception as e:
        return {"status": "error", "error_message": str(e)}
```

Step 2: Register it in server.py

```
# Add to TOOLS dict:
TOOLS = {
    ...
    "get_weather": weather.get_weather,
}

# Add to TOOL_DECLARATIONS list:
TOOL_DECLARATIONS = [
    ...
    {
        "name": "get_weather",
        "description": "Gets current weather for a city.",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "City name"}
            },
            "required": ["city"]
        }
    }
]
```

Step 3: Update the system prompt

Add a line to SYSTEM_PROMPT telling the model about the new capability: “You can check the weather for any city.” Gemini uses the system prompt + tool declarations together to decide when to call tools.

That’s It

No other changes needed. The execute_tool() function in server.py already handles dispatching any function in the TOOLS dict. Gemini will automatically start calling your new tool when the conversation warrants it.

10. What to Tell Claude Code

If you want to build a voice agent from scratch using Claude Code, here's the prompt. Adjust the product/brand details and tools for your use case.

The Prompt

```
Build me a real-time voice agent using Google Gemini 3.1 Flash Live.

Architecture:
- Python FastAPI server as a WebSocket relay between browser and Gemini
- Use the google-genai Python SDK (not raw WebSockets from the browser)
- Model: gemini-3.1-flash-live-preview
- Audio: PCM16, 16kHz mono input / 24kHz mono output
- CRITICAL: Use send_realtime_input(audio=blob) not media=blob

Server (FastAPI):
- /ws WebSocket endpoint
- Two async tasks: browser_to_gemini() and gemini_to_browser()
- Run both with asyncio.gather()
- Load GOOGLE_API_KEY from .env
- Serve static files from public/
- System prompt with [YOUR AGENT KNOWLEDGE]
- [Optional: tool declarations + TOOLS dict + execute_tool()]

Browser protocol:
- Send: {type: "audio", data: "<base64 pcm16>"}
- Receive: audio, status, turn_complete, interrupted, error

Audio in browser:
- Mic: getUserMedia at 16kHz, convert Float32->Int16->base64
- Playback: decode base64->Int16->Float32, schedule at 24kHz
- clearQueue() for instant barge-in (disconnect/reconnect gain node)
- All AudioContext creation inside click handler (autoplay policy)
```

Common Mistakes AI Agents Make (Without These Instructions)

- Connecting directly from browser to Gemini via raw WebSocket — unreliable, protocol issues
- Using the wrong model ID — must be one that supports bidiGenerateContent
- Using media= instead of audio= in send_realtime_input — deprecated, 3.1 rejects it
- Not handling barge-in — audio keeps playing when user tries to interrupt
- Creating AudioContext at page load — browsers block this, must be inside a click handler
- Putting systemInstruction in the setup message with audio-only responseModalities — known to hang on some models

11. Deployment to Production

Both demos need a persistent server (not serverless) because the WebSocket stays open for the entire conversation. Vercel, Netlify, and Lambda won't work.

Platform	Starting Price	WebSocket Support	Effort
Railway	$5/mo (included credits)	Yes	Low — connect repo, done
Render	$7/mo	Yes	Low — connect repo, set start cmd
Fly.io	~$2-5/mo	Yes	Medium — needs Dockerfile
DigitalOcean	$4-6/mo (droplet)	Yes	Medium — manage yourself

Railway Deploy (Easiest)

33. Push your project to GitHub
34. Create a new Railway project from your repo
35. Set environment variables: GOOGLE_API_KEY (and CLICKUP_API_KEY for Aria)
36. Set start command: uvicorn server:app --host 0.0.0.0 --port $PORT
37. Deploy — you get a public URL with HTTPS + WSS automatically

Security Note

Add rate limiting or authentication to the /ws endpoint in production. Without it, anyone can open voice sessions and burn your API credits.

12. Known Issues & Tips

Known Gemini Live API Issues

Issue	Details	Workaround
WebSocket 1011 errors	Random timeouts during tool execution	Retry; keep sessions under 10 min
systemInstruction hang	Setup hangs with audio-only + system prompt on some models	Use google-genai SDK (handles it correctly)
Model not found (1008)	Model doesn't support bidiGenerateContent	Use gemini-3.1-flash-live-preview
No response to audio	Some inputs get no response	Ensure PCM16 at exactly 16kHz mono
Config locked after connect	Can't change model/tools/prompt mid-session	Disconnect and reconnect

Tips

- Use headphones during testing to avoid echo/feedback.
- Browser DevTools Console shows all WebSocket messages — check there first when debugging.
- Server terminal shows Gemini SDK errors — watch both browser and server logs.
- Sessions expire after 15 minutes. Click mic again to reconnect.
- The system prompt is in server.py. Edit it to change personality or knowledge.
- To change the voice, modify the speechConfig in LiveConnectConfig. Voices: Kore, Puck, Charon, Fenrir, Aoede.
- For lowest latency, deploy near Google's API endpoints (us-central1).
- The Aria demo generates the system prompt at import time (captures current date/time). Restart the server to update the time context.
- All tool functions are synchronous (using requests). For slow APIs, consider running them in a thread executor to avoid blocking.